TD1 : de C au C++

Exercice1 : les bases de C++

1. Ecrire le programme C++ qui :
    a. Demande à l'utilisateur de saisir deux nombres de type double.
    b. Affiche le résultat des opérations arithmétiques +, -, *, / appliquées sur ces deux nombres (traitez le cas où la division est impossible).
    c. Affiche le reste de la division entière (%), après avoir converti les deux réels en entiers avec **static_cast<int>**
    d. Affiche la racine carrée des deux nombres (si le nombre est négatif, donnez la racine carrée de sa valeur absolue)
    e. Affiche le maximum, le minimum entre les deux nombres
    f. Affiche la puissance d'un nombre par l'autre.
2. Ajoutez un menu à choix multiples permettant à l'utilisateur de choisir une opération parmi les opérations listées ci-dessus (Utilisez l'instruction **switch … case**)
3. Utiliser une boucle **do … while** pour permettre à l'utilisateur d'effectuer plusieurs opérations tant qu'il le souhaite, sans quitter le programme.

Exercice 2 : Passage par référence

Soit P(x) un polynôme de degré 2 défini par :

$$P(x) = ax^2 + bx + c$$

Où a, b, et c sont les coefficients de P(x), avec $a \neq 0$

On souhaite résoudre l'équation P(x) = 0 pour trouver les racines du polynôme P(X), Pour cela :

1. Définir une structure **Polynome2D** contenant trois champs de type double ; a, b et c.
2. Définir une fonction **void saisie_polynome (Polynome2D & P)** ; qui permet de saisir les valeurs des coefficients a, b, et c d'un polynôme.
3. Définir une fonction **void racines_polynome** (**const Polynome2D & P**) ; qui résout l'équation :
   P(x) = 0 et affiche les racines du polynôme.
4. Écrire un programme principal **main** qui :

- Déclare une variable **poly** de type **Polynome2D,**
- Saisit les coefficients du polynôme **poly,**
- Affiche les racines du polynôme **poly,**

Exercice 3 : Surcharge de fonctions

1. Définir une fonction **bool rechercher (double * tab, int taille, double e) ;** qui renvoie **true** si l'élément **e** existe, **false** sinon.
2. Définir une fonction **void rechercher (double * tab, int taille, double e) ;** qui recherche un élément dans le tableau puis affiche le message « Elément trouvé » si l'élément existe ou « Elément non trouvé » sinon.
   ➔ Expliquez pourquoi la surcharge de la fonction rechercher ne sera pas acceptée dans la question n°2
3. Définir une fonction **double rechercher (double * tab, int taille, int pos) ;** qui retourne l'élément se trouvant à une position donnée dans le tableau.
   ➔ Expliquez pourquoi la surcharge de la fonction rechercher a été acceptée dans la question n°3

Exercice 4 : Gestion dynamique de la mémoire avec new et delete

Ecrire un programme qui permet de :

- Saisir un entier **n** pour stocker le nombre d'élèves d'une classe.
- Allouer dynamiquement de la mémoire pour un tableau de n éléments pour stocker les notes des élèves de la classe.
- Saisir les notes des n élèves et les stocker dans le tableau des notes.
- Calculer et afficher la moyenne de la classe.
- Afficher la note maximale et la note minimale de la classe.
- Stocker dans un nouveau tableau les notes des élèves triées en ordre décroissant.

Remarque : n'oubliez pas de libérer la mémoire allouée dynamiquement

Exercice 5 :

1. Créer une structure Contact qui contient les champs Nom (chaîne de caractères), Numéro (chaîne de caractères), et Email (chaîne de caractères).
2. Allouer dynamiquement un tableau de contacts d'une taille initiale donnée par l'utilisateur.
3. Implémenter des fonctions pour :
    - Ajouter un contact.
    - Afficher tous les contacts.
    - Rechercher un contact par nom.
    - Supprimer un contact.